

VDOO: Vinegar Diagonal Oil Oil Specification v1.1

Submitters

- Supriya Adhikary — Indian Institute of Technology Kanpur
- Cyril Bravo — LRE, EPITA
- Pilar Coscojuela-Escanilla — LRE/EPITA & Sorbonne University
- Anindya Ganguly — Indian Institute of Technology Kanpur
- Angshuman Karmakar — IIT Kanpur & Qryptshield Technologies Private Limited
- Suparna Kundu — COSIC, KU Leuven
- Debranjana Pal — The LNM Institute of Information Technology
- Ludovic Perret — LRE/EPITA & Sorbonne University
- Nitin Saxena — Indian Institute of Technology Kanpur

Website: <https://sites.google.com/view/vdoo-sign>

Primary Contact

Angshuman Karmakar
Contact no: (+91) 8967827714
Email: angshuman@cse.iitk.ac.in
IIT Kanpur & Qryptshield Technologies Private Limited
406, Rajeev Motwani Building,
Department of Computer Science and Engineering,
IIT Kanpur, Kalyanpur, India, Uttar Pradesh, 208016

Alternative Contact

Ludovic Perret
Sorbonne University
CNRS/LIP6/PolSys
Boîte courrier 169
4, Place Jussieu
P: +33-6-44-27-88-35
email : ludovic.perret@lip6.fr
F-75252 Paris cedex 5, France

Date: 30th June 2026

Contents

1	Introduction	4
2	Design Rationale	4
3	Feature Statements	4
3.1	Efficiency	5
3.2	Resistance to existing attacks	5
3.3	Small signature size	5
4	Our Proposal: VDOO Signature Scheme	6
4.1	Our Parameters	6
4.2	VDOOSetup: Generate Parameters	6
4.3	VDOO Central Polynomial Map and Inversion	7
4.4	VDOOKeyGen: VDOO Key Generation	8
4.5	VDOOSign: VDOO Signature Generation	8
4.6	VDOOVerif: VDOO Verification	9
4.7	Key Size Computation	9
4.8	Subspace Description of VDOO Central Polynomial	10
4.9	Optimized Key Generation Algorithm	11
5	Algebraic Security of VDOO	14
5.1	Direct Attack on VDOO	14
5.2	Simple Attack on VDOO	14
5.3	Rectangular Min-rank Attack on VDOO	16
5.4	Kipnis-Shamir Attack on VDOO	17
5.5	Intersection Attack on VDOO	18
5.6	Quantum Attacks	18
5.7	Provable security: EUF-CMA Security	19
5.8	Other Security Properties	20
6	Parameters Selection	20

7	Implementation Details	21
7.1	Finite Field Representation	21
7.2	Symmetric Primitives	21
7.3	Key Generation	21
7.4	Signature Generation	22
7.5	Verification	23
7.6	NGCC API Functions	23
7.7	Known Answer Test Values	23
8	Performance of VDOO	23
8.1	Results	24
8.2	Discussion	24
8.3	Limitations	25
	References	26

1 Introduction

Multivariate cryptography relies on the intractability of finding a solution to a system of multivariate quadratic polynomials in the finite field $\mathbb{F}_q$ (MQ problem). It is well known that this problem is NP-hard [30], making multivariate schemes post-quantum. Numerous schemes, such as Matsumoto-Imai encryption scheme [35], HFE/GeMMS[39, 11], Oil-Vinegar [40] signature, Rainbow [15] signature, Triangular [37, 45, 51] signature, Simple Matrix encryption [47], and MAYO [10], have been developed based on multivariate cryptography. Patarin first proposed the Oil-Vinegar signature [40]. A successful forgery attack was shown by Kipnis and Shamir [32] against this scheme. Further, Kipnis, Patarin, and Goubin upgraded the signature scheme by proposing Unbalanced Oil-Vinegar (UOV) [31].

2 Design Rationale

In this specification document, we present a comprehensive design specifications and theoretical framework of our proposal:

- We introduce Vinegar-Diagonal-Oil-Oil (VDOO), a novel multivariate signature scheme based on the framework proposed by Ganguly et al. [26] to enhance the efficiency and security of digital authentication. To the best of our knowledge, this is the first construction to integrate a diagonal layer within the Unbalanced Oil and Vinegar (UOV) framework, demonstrating that such a design improves performance without compromising security margins.
- Developed in alignment with the standards established by the Institute of Commercial Cryptography Standards (ICCS) and the Next-generation Commercial Cryptographic Algorithms Program (NGCC), our scheme provides a robust and scalable solution for post-quantum infrastructure. We propose specific parameter sets to meet diverse requirements: VDOO-128 (128-bit classical, 64-bit quantum security), VDOO-256 (256-bit classical, 128-bit quantum security), and VDOO-512 for high-security applications (512-bit classical, 256-bit quantum security).
- We establish that VDOO effectively withstands all known cryptanalytic attacks and formally outline its EUF-CMA security in Section 4. Through meticulous parameter selection, our findings reveal that the scheme achieves a remarkably compact signature size of 96 bytes at the 128-bit security level. This contrasts favorably with NIST-standardized signatures such as CRYSTALS-Dilithium [17], Falcon [22], and SPHINCS+ [2] (see Sections 5 and 6).

3 Feature Statements

VDOO is a new layer-based construction, which has one diagonal layer and then two UOV layers. We are adding each new variable in the central polynomial one by one diagonally. This offers efficiency. This translates to a reduction of the Gaussian elimination ($\text{GE}_{(q,n)}$)¹ which is the major computational bottleneck in the signature generation process. Suppose $x_1, x_2, \dots, x_v, x_{v+1}, \dots, x_{v+d}, \dots, x_{v+d+o_1}, \dots, x_{v+d+o_1+o_2}=:n$ are n variables defined over $\mathbb{F}_q$. In our construction, we call first v -variables as *vinegar variables*, next d -variables as *diagonal variables*, then next o_1 variables are *first-layer oil variables*, and

¹ $\text{GE}_{(q,n)}$: Gaussian elimination on a linear system with n unknowns and n linear equations over $\mathbb{F}_q$.

last o_2 variables are *second-layer oil variables*. Figure 1 illustrates the distribution of the variables in each layer of the VDOO central polynomial map.

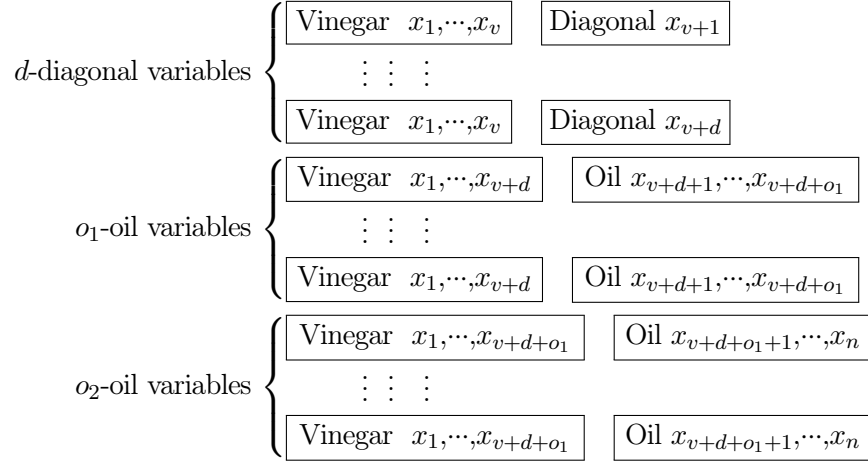


Figure 1: Variables in each layer of the VDOO central map

3.1 Efficiency

To thwart Beullens' simple attack [9], the authors of Rainbow increased the parameter set [27], which results in increasing the Gaussian elimination cost. The complexity of Gaussian elimination becomes approximately $o_1^3 + o_2^3$ where o_1 and o_2 are number of oil variables of Rainbow [15]. In our scheme, we adapt $d \approx (o_1 + o_2)/3$, $o'_1 \approx (o_1 + o_2)/3$, and $o'_2 \approx (o_1 + o_2)/3$ as the new parameters. This adjustment results in a Gaussian elimination complexity of around $o_1'^3 + o_2'^3$.

3.2 Resistance to existing attacks

We comprehensively analyze all possible attacks on multivariate cryptographic schemes against our scheme. In an attempt to recover diagonal variables, potential attackers begin by eliminating the uppermost oil layers. Beullens proposed method [9] facilitates the removal of these layers, aiding attackers. For instance, in order to compromise our round-one parameter set, a straightforward attack necessitates 2^{134} -field operations. Furthermore, Beullens combined this simple attack with the rectangular min-rank attack [8, 9]. In line with previous efforts, we execute this combined attack against our scheme, determining that it requires 2^{138} field operations to break the SL-1 parameter set. Additionally, we conduct the intersection attack and the direct attack on our scheme, both of which exhibit complexities exceeding 2^{134} -field operations. Consequently, these references collectively imply that VDOO appears to withstand all known attacks securely. We also outline the EUF-CMA security of the VDOO scheme.

3.3 Small signature size

We present multiple parameters that can withstand the aforementioned attacks. Specifically, our level-one parameters that can provide 128-bit classical and at least 80-bit post-quantum security have a signature

size of 85 bytes (B) and public-key size of 323.1 kilobytes (KB) (further elaborated in Table 1). This is the smallest signature size among the majority of all multivariate signature schemes (for additional insights, refer to Tables 1).

4 Our Proposal: VDOO Signature Scheme

In our scheme, we introduce a new design element called *diagonals* into the Oil-Vinegar scheme. Let, $\mathbf{x} \in \mathbb{F}_q^n$, we pick the first v variables as vinegar variables. We denote the next d variables as diagonal variables. In this layer, we introduce d quadratic equations. In any i -th ($1 \leq i \leq d$) equation, only $v+i$ -th variable is unknown among $v+i$ variables. In the following layers, we apply the Oil-Vinegar technique. This means we can generate o_1 OV polynomials using $v+d$ -vinegar variables and newly added o_1 -oil variables. Further, we construct o_2 OV polynomials using $v+d+o_1$ -vinegar variables and newly added o_2 -oil variables. Finally, we have a quadratic system with $n=v+d+o_1+o_2$ variables and $m=d+o_1+o_2$ homogeneous quadratic equations.

4.1 Our Parameters

These parameters are used to design our construction VDOO:

- q : Size of the finite field.
- v : Number of vinegar variables.
- d : Number of diagonal variables.
- o_1 : Number of first-layer oil variables.
- o_2 : Number of second-layer oil variables.
- n : Total number of variables, where $n=v+d+o_1+o_2$.
- m : Total number of equations, where $m=d+o_1+o_2$.

4.2 VDOOSetup: Generate Parameters

To construct polynomial maps we need to define parameters associated with this. In this phase algorithm takes input the security parameter λ and output the parameter tuple, that is $\text{params}=(q, v, d, o_1, o_2) \leftarrow \text{VDOOSetup}(1^\lambda)$. Here,

- Finite field $\mathbb{F}_q$ which has q elements.
- Positive integers v, d, o_1 , and o_2 , where v denotes the number of vinegar variables, d is the number of diagonal variables, o_1 and o_2 stands for the number of first and second layer oil variables respectively. Therefore, total number of variables is $n=v+d+o_1+o_2$, and number of equations is $m=d+o_1+o_2$.

4.3 VDOO Central Polynomial Map and Inversion

Construction of central polynomial map $\mathcal{F}: \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ plays an important role in the multivariate signature schemes. To the best of our knowledge, we are the first to propose a central polynomial map that involves vinegar, diagonal, and oil variables in a three-layer construction.

- **Diagonal Layer.** Here, we explain the structure of any central polynomial f_k for the diagonal layer $k \in [v+1:v+d]$. Each f_k is defined as follows.

$$f_{k-v}(x_1, x_2, \dots, x_n) = \sum_{i=1}^{k-1} \alpha_{i,k}^{(k)} x_i x_k + \sum_{i,j=1, i \leq j}^{k-1} \beta_{i,j}^{(k)} x_i x_j$$

Each coefficient $\alpha_{i,j}^{(k)}$, and $\beta_{i,j}^{(k)} \in_U \mathbb{F}_q$. The subroutine $\text{DiagPoly}(q, k)$ is used to generate such central polynomial f_k in the diagonal layer.

- **First Oil Layer.** In this oil layer, we use $v+d$ variables as vinegar variables and next o_1 variables as oil variables. All these variables help us to construct o_1 homogeneous quadratic polynomials of the following form.

$$f_{k-v}(x_1, x_2, \dots, x_n) = \sum_{i=1}^{v+d} \sum_{j=1}^{v+d} \alpha_{i,j}^{(k)} x_i x_j + \sum_{i=1}^{v+d} \sum_{j=v+d+1}^{v+d+o_1} \beta_{i,j}^{(k)} x_i x_j$$

where $k \in [v+d+1:v+d+o_1]$, $\alpha_{i,j}^{(k)}$, and $\beta_{i,j}^{(k)} \in_U \mathbb{F}_q$.

- **Second Oil Layer.** The topmost oil layer has $v+d+o_1$ vinegar and o_2 oil variables. That means, it has o_2 quadratic equations. Those equations are of the form

$$f_{k-v}(x_1, x_2, \dots, x_n) = \sum_{i=1}^{v+d+o_1} \sum_{j=1}^{v+d+o_1} \alpha_{i,j}^{(k)} x_i x_j + \sum_{i=1}^{v+d+o_1} \sum_{j=v+d+o_1+1}^{v+d+o_1+o_2} \beta_{i,j}^{(k)} x_i x_j,$$

where $k \in [v+d+o_1+1:v+d+o_1+o_2=n]$ and $\alpha_{i,j}^{(k)}$, and $\beta_{i,j}^{(k)} \in_U \mathbb{F}_q$. We denote this as $\text{OVPoly}(q, v, o)$ to generate an oil-vinegar central polynomial with v vinegar variables and o oil variables.

Here, Algorithm 1, uses OVPoly and DiagPoly to generate a VDOO central map $\mathcal{F}$.

Algorithm 1 VDOOCentPoly

Require: Parameter tuple $params = (q, v, d, o_1, o_2)$

Ensure: Central map $\mathcal{F} = (f_1, \dots, f_m) : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$

- 1: Compute $m = d + o_1 + o_2$ and $n = v + m$.
 - 2: **for** $1 \leq i \leq d$
 - 3: $f_i \leftarrow \text{DiagPoly}(q, i)$
 - 4: **for** $d+1 \leq i \leq d+o_1$
 - 5: $f_i \leftarrow \text{OVPoly}(q, v+d, o_1)$
 - 6: **for** $d+o_1+1 \leq i \leq m$
 - 7: $f_i \leftarrow \text{OVPoly}(q, v+d+o_1, o_2)$
 - 8: **Return** VDOO central polynomial $\mathcal{F}$
-

Inversion. The main computational bottleneck of UOV-based constructions is the inversion of the central polynomial. It requires Gaussian elimination which runs in $O(N^3)$. However, in our scenario inversion of the diagonal polynomials is straightforward as there is only one unknown variable. Nevertheless, the inversion of OV polynomials in the remaining two layers each needs a Gaussian elimination. Therefore, inverting VDOO central polynomial map needs two Gaussian elimination only. This is shown in Algorithm 2. Following two algorithms help to compute the inverse of the VDOO central polynomial.

ST The subroutine *Substitution* or ST converts a bunch of oil-vinegar polynomials to a bunch of linear polynomials consists of oil variables by fixing the vinegar variables. That means, ST substitutes vinegar variables $x_1, \dots, x_v$ by random values (in $\mathbb{F}_q$) in the bunch of o oil-vinegar polynomials $(f_i)_{i=1}^o$ and converts it to a bunch of linear polynomials of o oil variables $(\tilde{f}_i)_{i=1}^o$.

GE The $\text{GE}_{(q,l)}$ denotes Gaussian elimination for l unknowns over the linear system of equations $(\tilde{f}_i = y_i)_{i=1}^l$ over $\mathbb{F}_q$. It returns a failure when the rank of the matrix representing the linear system is less than l .

Algorithm 2 VDOOCentPoly_Inversion

Require: Central map: $\mathcal{F} = (f_1, \dots, f_m) : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ and $\mathbf{y} \in \mathbb{F}_q^m$, and params.

Ensure: A vector $\mathbf{x} \in \mathbb{F}_q^n$ such that $\mathcal{F}(\mathbf{x}) = \mathbf{y}$.

- 1: $m \leftarrow d + o_1 + o_2$ and $n \leftarrow v + m$
 - 2: Randomly fix first v -vinegar variables $x_1, \dots, x_v \leftarrow_{\$} \mathbb{F}_q$
 - 3: **for** $1 \leq i \leq d$
 - 4: compute x_{v+i} using $y_i, x_1, \dots, x_{v+i-1}$ and f_i .
 - 5: $(\tilde{f}_{d+1}, \dots, \tilde{f}_{v+d}) \leftarrow \text{ST}(f_{d+1}(x_1, \dots, x_{v+d}), \dots, f_{d+o_1}(x_1, \dots, x_{v+d}))$
 - 6: $(x_{v+d+1}, \dots, x_{v+d+o_1}) \leftarrow \text{GE}_{(q,o_1)}(\tilde{f}_{d+1} = y_{d+1}, \dots, \tilde{f}_{d+o_1} = y_{d+o_1})$.
 - 7: $(\tilde{f}_{d+o_1+1}, \dots, \tilde{f}_m) \leftarrow \text{ST}(f_{d+o_1+1}(x_1, \dots, x_{n-o_2}), \dots, f_m(x_1, \dots, x_{n-o_2}))$
 - 8: $(x_{v+d+o_1+1}, \dots, x_n) \leftarrow \text{GE}_{(q,o_2)}(\tilde{f}_{d+o_1+1} = y_{d+o_1+1}, \dots, \tilde{f}_m = y_m)$
 - 9: **Return** $\mathbf{x} \in \mathbb{F}_q^n$
-

4.4 VDOOKeyGen: VDOO Key Generation

The VDOOKeyGen in Algorithm 3 generates two random invertible affine maps $\mathcal{S} : \mathbb{F}_q^m \rightarrow \mathbb{F}_q^m$ and $\mathcal{T} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$ along with the VDOO-central map $\mathcal{F} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$. Here, *secret/signing key* is $\mathcal{S}, \mathcal{F}$, and $\mathcal{T}$ and *public/verification key* is the composition map $\mathcal{P}$, where $\mathcal{P} = \mathcal{S} \circ \mathcal{F} \circ \mathcal{T} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$. Note that, the individual information of secret maps allows user to compute the inverse of $\mathcal{P}$ efficiently. We denote $S \leftarrow \text{randomMatrix}(q, m, \text{seed})$ to generate a random $m \times m$ matrix over $\mathbb{F}_q$ from a *seed*, $\text{invMat}(q, m, S)$ helps to compute the inverse of a $m \times m$ matrix S over $\mathbb{F}_q$, and $\text{Affine}(S, \mathbf{a})$ computes $\mathcal{S} \leftarrow S \cdot \mathbf{x} + \mathbf{a}$.

4.5 VDOOSign: VDOO Signature Generation

Similar to the other OV based constructions [10, 14, 31, 15], we use the hash-and-sign paradigm for our signature algorithm as shown in Algorithm 4. We use a hash function $\mathcal{H} : \{0,1\}^* \rightarrow \mathbb{F}_q^m$. Signer knows each polynomial map, so it can compute the inverse of each map *i.e.* $\mathcal{S}^{-1}$, $\mathcal{F}^{-1}$, and $\mathcal{T}^{-1}$. If GE reports a failure during the computation of $\mathcal{F}^{-1}$, we restart the process by regenerating the salt and repeating the entire procedure. Finally, the signature is computed as $\mathcal{P}^{-1}(\mathcal{H}(\mathcal{H}(\text{msg}) || \text{salt}))$.

Algorithm 3 VDOOKeyGen

Require: Parameter tuple params .

Ensure: Generate public and private key pair.

- Public key: $\text{pk} = \mathcal{P}$.
 - Secret key: $\text{sk} = \mathcal{S}, \mathcal{T}$, and $\mathcal{F}$.
- 1: $m \leftarrow d + o_1 + o_2$ and $n \leftarrow m + v$
 - 2: $\text{seed} \leftarrow \text{PRNG}(1^\lambda)$ $\triangleright \lambda$ is the security parameter
 - 3: **while** $(\det(S) \neq 0 \ \&\& \ \det(T) \neq 0)$ **do**
 - 4: $S \leftarrow \text{randomMatrix}(q, m, \text{seed})$ $\triangleright S \in_U \mathbb{F}_q^{m \times m}$
 - 5: $T \leftarrow \text{randomMatrix}(q, n, \text{seed})$ $\triangleright T \in_U \mathbb{F}_q^{n \times n}$
 - 6: **end while**
 - 7: $\mathbf{a} \in_U \mathbb{F}_q^m$ and $\mathbf{b} \in_U \mathbb{F}_q^n$ $\triangleright$ generate two random vector
 - 8: $\text{inv}S \leftarrow \text{invMat}(q, m, S)$ and $\text{inv}T \leftarrow \text{invMat}(q, n, T)$ $\triangleright$ compute inverse of matrices
 - 9: $\mathcal{S} \leftarrow \text{Affine}(S, \mathbf{a})$ and $\mathcal{T} \leftarrow \text{Affine}(T, \mathbf{b})$ $\triangleright$ Constructing invertible affine maps
 - 10: $\mathcal{F} \leftarrow \text{VDOOCentPoly}(\text{params})$ $\triangleright$ generate VDOO central map
 - 11: Compute $\mathcal{P} \leftarrow \mathcal{S} \circ \mathcal{F} \circ \mathcal{T}$
 - 12: **Return** $\text{pk} = \mathcal{P}$ and $\text{sk} = (\text{inv}S, \mathbf{a}, \text{inv}T, \mathbf{b})$ (equivalently sending $\mathcal{S}$, and $\mathcal{T}$).
-

Algorithm 4 VDOOSign

Require: $\text{sk} = (\text{inv}S, \mathbf{a}, \text{inv}T, \mathbf{b})$, message msg , and $\mathcal{H}: \{0,1\}^* \rightarrow \mathbb{F}_q^m$

Ensure: a signature $\sigma = (\mathbf{s}, \text{salt})$

- 1: $\text{salt} \leftarrow \text{PRNG}$
 - 2: Use hash function $\mathbf{d} \leftarrow \mathcal{H}(\mathcal{H}(\text{msg}) \parallel \text{salt})$
 - 3: Compute $\mathbf{t} = \text{inv}S \times (\mathbf{d} - \mathbf{a})$ $\triangleright \mathbf{t} = \mathcal{S}^{-1}(\mathbf{d})$
 - 4: Compute $\mathbf{y} = \mathcal{F}^{-1}(\mathbf{t})$ using VDOOCentPoly_Inversion 2.
 - 5: Compute $\mathbf{s} = \text{inv}T \times (\mathbf{y} - \mathbf{b})$ $\triangleright \mathbf{s} = \mathcal{T}^{-1}(\mathbf{y})$
 - 6: **Return** signature $\sigma = (\mathbf{s}, \text{salt})$
-

Efficiency analysis. As mentioned earlier, the major computational overhead of OV-based schemes is the Gaussian elimination procedure. In VDOO, during signing, we have to compute only one Gaussian elimination *i.e.* computation of $\mathcal{F}^{-1}$. The computation of $\mathcal{S}^{-1}$ and $\mathcal{T}^{-1}$ can be done during the key-generation procedure.

4.6 VDOOVerif: VDOO Verification

Our verification procedure is simple. It needs a polynomial evaluation of $\mathcal{P}$, requiring just $O(N^3)$ field operations. Compute $\mathbf{d}' = \mathcal{P}(\mathbf{s})$ from public key $\mathcal{P}$ and signature $\sigma = (\mathbf{s}, \text{salt})$. The signature is accepted if $\mathbf{d}' = \mathcal{H}(\mathcal{H}(\text{msg}) \parallel \text{salt})$ holds, else rejected.

4.7 Key Size Computation

Our VDOO contains one diagonal layer and two UOV layers. The size of the private key is determined first, followed by the size of the public key.

Algorithm 5 VDOOVerif

Require: $\text{pk}=\mathcal{P}$; message msg ; signature $\sigma=(\mathbf{s},\text{salt})$ and $\mathcal{H}:\{0,1\}^* \rightarrow \mathbb{F}_q^m$.

Ensure: *accept* or *reject*

- 1: Use hash function to compute $\mathbf{d} \leftarrow \mathcal{H}(\mathcal{H}(\text{msg}) \parallel \text{salt})$
 - 2: Compute $\mathbf{d}' = \mathcal{P}(\mathbf{s})$
 - 3: **if** $\mathbf{d} = \mathbf{d}'$ **then** output *accept*
 - 4: **else** *reject*
 - 5: **end if**
 - 6: **Return** *accept* or *reject*
-

- Size of the central map $\mathcal{F}$ for a diagonal layer having d -diagonal polynomials is $\sum_{i=1}^d \left(\frac{v_i(v_i+1)}{2} + v_i \right)$ field elements. The first diagonal layer has $v_1 = n - m$ vinegar variables. In any diagonal layer, a central polynomial f_i has v_i vinegar variables and f_{i+1} -th polynomial has $v_{i+1} = v_i + 1$ vinegar variables.
- Size of the central map $\mathcal{F}$ for a UOV layer is around $o \times \left(\frac{v(v+1)}{2} + ov \right)$ field elements. Such a UOV layer has v vinegar variables and o oil variables.

The sizes of the two affine transformations are as follows: for $\mathcal{S}$ we need $m(m+1)$, while for $\mathcal{T}$ we need $n(n+1)$, field elements. These maps can be generated using a random seed.

Now we are interested in computing the size of the public key of standard VDOO. Each n -variate quadratic polynomial requires $\frac{(n+1)(n+2)}{2}$ field elements. Therefore, the size of the public key is $m \frac{(n+1)(n+2)}{2}$. Further optimization of the public key is possible [42, 43]. It optimized the public key size from $O(mn^2 \log q)$ to $O(m^3 \log q)$.

4.8 Subspace Description of VDOO Central Polynomial

Our scheme can be explained through Beullens's subspace descriptions [8]. This description is useful to understand the cryptanalysis of VDOO. In this case, we have $d+2$ input and output subspaces. These sequences of nested subspaces are as follows.

- **Input subspaces** $\mathbb{F}_q^n \supset D_1 \supset D_2 \supset \dots \supset D_d \supset O_1 \supset O_2$.
- **Output subspaces** $\mathbb{F}_q^m \supset Q_{1,1} \supset Q_{1,2} \supset \dots \supset Q_{1,d} \supset Q_2 \supset Q_3 = \{0\}$.

In the Figure 2 (single arrow denotes $\mathcal{P}$ and bold arrow denotes $\mathcal{DP}(\mathbf{x}, \cdot)$), these following relations will hold: $\dim(D_i) = \dim(D_{i+1}) + 1$ and $\dim(Q_{1,i}) = \dim(Q_{1,i+1}) + 1$ for $1 \leq i < d$. Also, $\dim(D_1) = m$, $\dim(D_i) = \dim(Q_{1,i-1})$ for $1 < i \leq d$. In addition, $\dim(O_1) = \dim(Q_{1,d}) = o_1 + o_2$, $\dim(O_2) = \dim(Q_2) = o_2$.

The signer first fixes $\mathbf{v} \in_U \mathbb{F}_q^n$. Since $\dim(\tilde{D}_i) = \dim(D_i) - \dim(D_{i+1}) = 1$, so for diagonal layer computing $\mathbf{d}_1, \dots, \mathbf{d}_d$ is very easy. Once these vectors are found, then update $\mathbf{v} \leftarrow \mathbf{v} + \mathbf{d}_1 + \dots + \mathbf{d}_d$. Now, signer needs to solve for $\tilde{\mathbf{o}}_1 \in \tilde{O}_1 (= O_1/O_2)$, so that the following relation holds. Note that, $\dim(\tilde{O}_1) = o_1$.

$$\mathcal{P}(\mathbf{v}) + \mathcal{DP}(\mathbf{v}, \tilde{\mathbf{o}}_1) = \mathbf{t} \pmod{Q_2}.$$

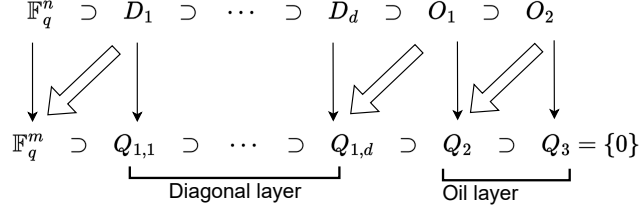


Figure 2: Central polynomial of VDOO

We know that the above equation is a linear system of o_1 variables and o_1 equations. With the probability $(1-1/q)$, the signer will be able to compute $\mathbf{o}_1$. Then the signer again updates $\mathbf{v} \leftarrow \mathbf{v} + \mathbf{o}_1$ and follows a similar strategy to find $\mathbf{o}_2 \in O_2$. Thus the signer can finally compute the pre-image of $\mathbf{t}$.

4.9 Optimized Key Generation Algorithm

The affine transformations $\mathcal{T} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$ and $\mathcal{S} : \mathbb{F}_q^m \rightarrow \mathbb{F}_q^m$ are represented by the structured block upper-triangular matrices

$$T = \begin{pmatrix} I_v & T^{(1)} & T^{(2)} & T^{(3)} \\ 0 & I_d & T^{(4)} & T^{(5)} \\ 0 & 0 & I_{o_1} & T^{(6)} \\ 0 & 0 & 0 & I_{o_2} \end{pmatrix}, \quad \text{and } S = \begin{pmatrix} I_d & S'_1 & S'_2 \\ 0 & I_{o_1} & S'_3 \\ 0 & 0 & I_{o_2} \end{pmatrix}, \quad (1)$$

where every off-diagonal block is sampled uniformly from $\mathbb{F}_q$. Since both T and S are upper-triangular matrices, their determinants satisfy $\det(T) = \det(S) = 1$. Consequently, both transformations are invertible, and their inverses can be computed efficiently using block back-substitution. This structured choice significantly reduces the cost of public-key generation while preserving the correctness of the affine transformations.

Computing $Q = T^\top F T$ Instead of directly computing the public map, we first evaluate $Q^{(k)} = T^\top F^{(k)} T$ for every central polynomial $F^{(k)}$. Here $F^{(k)}$ is the matrix representation of the homogeneous quadratic polynomial f_k . Since each layer of VDOO contains only a small subset of non-zero quadratic blocks, only the corresponding block products need to be evaluated. The diagonal layer contains only the vinegar-vinegar block together with a single vinegar-diagonal interaction, whereas the two oil layers additionally introduce vinegar-oil interactions while preserving the absence of oil-oil quadratic terms. Consequently, the matrix products inherit the sparsity of the central map and avoid the dense quadratic computations required by a generic transformation.

Diagonal layer. For $k \in [v+1, v+d]$, only the vinegar-vinegar block and the vinegar-diagonal interaction

are nonzero. Let $\Sigma_1 = F_1^{(k)} + (F_1^{(k)})^\top$. The transformed blocks are given by

$$\begin{aligned}
Q_1 &= F_1^{(k)}, \\
Q_2 &= \Sigma_1 T^{(1)} + F_2^{(k)}, \\
Q_3 &= \Sigma_1 T^{(2)} + F_2^{(k)} T^{(4)}, \\
Q_4 &= \Sigma_1 T^{(3)} + F_2^{(k)} T^{(5)}, \\
Q_5 &= \text{UT} \left((T^{(1)})^\top F_1^{(k)} T^{(1)} + (T^{(1)})^\top F_2^{(k)} \right), \\
Q_6 &= (T^{(1)})^\top \Sigma_1 T^{(2)} + (T^{(1)})^\top F_2^{(k)} T^{(4)} + (F_2^{(k)})^\top T^{(2)}, \\
Q_8 &= \text{UT} \left((T^{(2)})^\top F_1^{(k)} T^{(2)} + (T^{(2)})^\top F_2^{(k)} T^{(4)} \right), \\
Q_{10} &= \text{UT} \left((T^{(3)})^\top F_1^{(k)} T^{(3)} + (T^{(3)})^\top F_2^{(k)} T^{(5)} \right),
\end{aligned} \tag{2}$$

where Q_7 and Q_9 are obtained analogously from Q_6 and Q_8 by replacing $(T^{(2)}, T^{(4)})$ with $(T^{(3)}, T^{(5)})$.

Oil layer 1. For $k \in [v + d + 1, v + d + o_1]$, the transformed blocks $Q_1, \dots, Q_4$ retain the same form as in the diagonal layer, while the additional vinegar–oil interactions contribute to the remaining blocks. Let $\bar{F}_1$ denote the quadratic block over the complete vinegar variables, and define $\Sigma_{\bar{F}} = \bar{F}_1 + \bar{F}_1^\top$, and $\Sigma_5 = F_5^{(k)} + (F_5^{(k)})^\top$. The additional transformed blocks become

$$\begin{aligned}
Q_8 &= \text{UT} \left((T^{(2)})^\top \bar{F}_1 T^{(2)} + (T^{(2)})^\top F_4^{(k)} + F_5^{(k)} \right), \\
Q_9 &= (T^{(2)})^\top \Sigma_{\bar{F}} T^{(3)} + (T^{(2)})^\top F_4^{(k)} T^{(6)} + (T^{(2)})^\top F_5^{(k)} + (F_4^{(k)})^\top T^{(3)} + \Sigma_5 T^{(6)}, \\
Q_{10} &= \text{UT} \left((T^{(3)})^\top \bar{F}_1 T^{(3)} + (T^{(3)})^\top F_4^{(k)} T^{(6)} + (T^{(6)})^\top F_5^{(k)} T^{(6)} \right).
\end{aligned} \tag{3}$$

Oil layer 2. For $k \in [v + d + o_1 + 1, n]$, the vinegar variables additionally include the first oil layer. Consequently, the transformed blocks $Q_1, \dots, Q_8$ follow the same derivation as above with the enlarged vinegar set. Let $\bar{F}'_1$ denote the quadratic block over this complete vinegar space. The remaining block is

$$Q_{10} = \text{UT} \left((T^{(3)})^\top \bar{F}'_1 T^{(3)} + (T^{(3)})^\top F_7^{(k)} + (T^{(5)})^\top F_8^{(k)} + (\text{oil}_1\text{-cross terms involving } T^{(6)}) \right). \tag{4}$$

After computing every transformed polynomial, the resulting symmetric matrices are converted into their upper-triangular representation before being inserted into the Macaulay matrix M_Q , which is subsequently used to assemble the public key.

Public Key Assembly Let M_Q denote the Macaulay matrix formed from all transformed polynomials $Q^{(k)}$. Since the output transformation S is also block upper triangular, the public key can be assembled using only block matrix additions,

Algorithm 6 Efficient Key Generation of VDOO

Require: Linear maps T, S of form (1); matrices $F^{(k)}$, $k = v+1, \dots, n$

Ensure: Public key P (matrices $M_{P_1}, M_{P_2}, M_{P_3}$)

```

1: for  $k = v+1$  to  $v+d$  do                                ▷ diagonal layer
2:   Compute  $Q_1, Q_2, Q_3, Q_4, Q_5, Q_6, Q_7, Q_8, Q_{10}$  via (2)
3: end for
4: for  $k = v+d+1$  to  $v+d+o_1$  do                                ▷ oil layer 1
5:   Compute  $Q_1-Q_4$  via (2) (with  $F_3^{(k)}$ )
6:   Compute  $Q_8, Q_9, Q_{10}$  via (3)
7: end for
8: for  $k = v+d+o_1+1$  to  $n$  do                                ▷ oil layer 2
9:   Compute  $Q_1-Q_8$  by the pattern of (3)
10:  Compute  $Q_{10}$  via (4)
11: end for
12: for  $k = v+1$  to  $n$  do
13:  Insert blocks of  $Q^{(k)}$  into row  $(k-v)$  of  $M_Q$ 
14: end for
15:  $M_{P_1} \leftarrow M_{Q_1} + S'_1 M_{Q_2} + S'_2 M_{Q_3}$ 
16:  $M_{P_2} \leftarrow M_{Q_2} + S'_3 M_{Q_3}$ 
17:  $M_{P_3} \leftarrow M_{Q_3}$ 
18: return  $M_{P_1}, M_{P_2}, M_{P_3}$ 

```

$$\begin{aligned}
M_{P_1} &= M_{Q_1} + S'_1 M_{Q_2} + S'_2 M_{Q_3}, \\
M_{P_2} &= M_{Q_2} + S'_3 M_{Q_3}, \\
M_{P_3} &= M_{Q_3},
\end{aligned} \tag{5}$$

which is the direct three-layer analogue of Rainbow's efficient key generation. Only the upper blocks require additional computations, while the final block is copied unchanged. The complete procedure is summarized in Algorithm 6.

Key Size Analysis Algorithm 6 changes only the computation of the public key and does not modify the underlying key representation. Thus, the public key still contains m homogeneous quadratic polynomials in n variables, while the secret key consists of the structured affine maps together with the central map:

$$\begin{aligned}
|PK| &= m \cdot \frac{n(n+1)}{2}, \\
|SK| &= |S| + |T| + |F_i|.
\end{aligned} \tag{6}$$

Here $|PK|$ and $|SK|$ are measured in field elements, and only the free parameters of S and T need to be stored. The exact key sizes for all parameter sets are reported in Table 1.

5 Algebraic Security of VDOO

Cryptanalysis that targets solving the MQ problem directly, is known as the direct attack in multivariate cryptography [3, 18, 19, 6]. Later researchers have used the special structure of the quadratic system and improved the state-of-the-art, like, band-separation attack [16, 48, 46], intersection attack [8], and simple attack [9].

To determine the complexity of the attacks described below by the number of field multiplications required to perform the attack. One $\mathbb{F}_q$ -field multiplication needs $(2(\log_2 q)^2 + \log_2 q)$ gates. Here, each $2(\log_2 q)^2$ -bit stands for one $(\log_2 q)^2$ -bit multiplication (represented as AND gates) and the same number of additions (represented as XOR gates) during one $\mathbb{F}_q$ -multiplication. Additionally, $\log_2 q$ bits are needed for $\log_2 q$ -bit additions involved in one $\mathbb{F}_q$ -addition, which is required for each field multiplication that occurs during an attack. For example, the cost one $\mathbb{F}_{16}$ -multiplication requires 36 gates. Such a strategy to determine the complexity is standard and has been also followed in other MQ-based signature schemes [23, 10, 20].

Henceforth, in this document, we use the parameter set $(q, v, d, o_1, o_2) = (16, 67, 16, 14, 40)$ as an example to demonstrate the complexity of the following attacks. Incidentally, this is also our SL-1 parameter. Our full parameter set is given in Table. 1.

5.1 Direct Attack on VDOO

The direct attack is the fundamental methodology for forging any multivariate signature scheme. To counterfeit a VDOO signature, an attacker aims to solve an underdetermined system with n variables and m homogeneous equations ($n > m$), to find $\mathbf{s}$ such that $\mathcal{P}(\mathbf{s}) = \mathbf{t}$. The basic approach involves converting this underdetermined system into a determined one by fixing $n - m$ variables. Subsequently, quadratic system-solving techniques like the Wiedemann XL algorithm [49, 13] or Gröbner basis methods such as F4 or F5 [18, 19] are applied. Another approach named hybrid approach [6, 7] involves guessing k variables prior to solving the system. The time complexity of this attack, using the approach outlined in [6], is expressed in terms of field multiplications as:

$$\min_{0 \leq k \leq m-2} 3 \cdot q^k \cdot \binom{m-k+d}{d}^2 \binom{m-k}{2}$$

Here, k denotes the number of variables fixed during the algorithm, and d represents the smallest integer for which the coefficient of t^d in the series $\frac{(1-t^2)^m}{(1-t)^{m-k}}$ is non-positive.

Example for SL-1 parameters. Our level one parameter set has 137 variables and 70 constraints. According to [6], we fix 67 variables. Now, the complexity formula of the direct attack is minimized for $k = 14$ for which we get $d = 16$. The total complexity is around 2^{171} .

5.2 Simple Attack on VDOO

In 2022, Beullens proposed the *simple attack* against Rainbow [15]. For Rainbow, this highly effective attack reduces n -unknown and m -constraints in the quadratic system to $n - m$ -unknown and m -constraints. Now an attacker can apply the same methodology on VDOO to recover the secret key. Recall from Figure. 2, $\mathcal{P}$ is the public polynomial map, and sequences of nested input and output subspaces are,

- **Input subspaces** $\mathbb{F}_q^n \supset D_1 \supset D_2 \supset \dots \supset D_d \supset O_1 \supset O_2$.
- **Output subspaces** $\mathbb{F}_q^m \supset Q_{1,1} \supset Q_{1,2} \supset \dots \supset Q_{1,d} \supset Q_2 \supset Q_3 = \{0\}$.

The main crux of the simple attack lies in finding a vector within O_2 (as depicted in Figure. 2). To achieve this, the attacker must solve a quadratic system with $n-m$ unknowns and m constraints using the XL algorithm. This computational step constitutes the most significant component of the entire attack. Here is a step-by-step outline detailing the cryptanalysis of our scheme using the simple attack.

Input: Public polynomial map $\mathcal{P}$.

Output: Recover sequences of subspaces.

Find a vector $\mathbf{o} \in O_2$: Choose $\mathbf{v} \in_U \mathbb{F}_q^n$. Then from Figure. 2, $\mathcal{DP}_{\mathbf{v}} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ is a linear map, in particular it maps O_2 to Q_2 . The attacker uses this linear relation to reduce the number of unknowns present in the quadratic system. Therefore, to find a vector, an attacker should solve the following system.

$$\begin{aligned}\mathcal{DP}_{\mathbf{v}}(\mathbf{o}) &= 0 \\ \mathcal{P}(\mathbf{o}) &= 0\end{aligned}$$

With probability $\approx 1/q$, the attacker successfully guesses a vector in O_2 . Later, the attacker deploys the XL algorithm to solve the quadratic system of $n-m$ -unknowns and m -constraints. Thus attacker recovers $\mathbf{o}$.

Recover Q_2 : Attacker will retrieve Q_2 using the information $\mathbf{o} \in O_2$. Note that, $\mathcal{DP}_{\mathbf{o}} : O_2 \rightarrow Q_2$ is a linear map. Therefore,

$$\text{Span}\{ \mathcal{DP}_{\mathbf{o}}(\mathbf{e}_1), \dots, \mathcal{DP}_{\mathbf{o}}(\mathbf{e}_n) \} \subseteq Q_1$$

for some linearly independent vectors $\mathbf{e}_i$. For enough such $\mathbf{e}_i$'s equality will hold.

Recover O_2 : To recover O_2 , solve the following system of linear equations. Because with high probability kernel of $\mathcal{DP}_{\mathbf{o}}$ matches with O_2 .

$$\begin{aligned}\mathcal{DP}_{\mathbf{o}}(\mathbf{e}_1) &\equiv 0 \pmod{Q_2} \\ \mathcal{DP}_{\mathbf{o}}(\mathbf{e}_2) &\equiv 0 \pmod{Q_2} \\ &\vdots \\ \mathcal{DP}_{\mathbf{o}}(\mathbf{e}_n) &\equiv 0 \pmod{Q_2}\end{aligned}$$

Recover a vector $\mathbf{o}' \in O_1$: Now the quadratic system $\mathcal{P}$ reduces to $m' = m - o_2$ equations and $n' = n - o_2$ variables. To recover O_1 , the goal of the attacker is to find a vector in $\mathbf{o} \in O_1$. Again attacker will guess a vector $\mathbf{v}' \in \mathbb{F}_q^{n'}$. Like above, a similar argument shows that $\mathcal{DP}_{\mathbf{v}'} : O_1 \rightarrow Q_{1,1}$ is a linear map and the attacker tries to solve the following systems mod Q_2 .

$$\begin{aligned}\mathcal{DP}_{\mathbf{v}'}(\mathbf{o}') &= 0 \pmod{Q_2} \\ \mathcal{P}(\mathbf{o}') &= 0 \pmod{Q_2}\end{aligned}$$

The attacker runs the XL algorithm to solve the quadratic system of $n' - m'$ -unknowns and m' -constraints.

Recover O_1 : Attacks follows same approach as recovering O_2 to recover O_1 . Here, an attacker solves a system $\mathcal{DP}_{\sigma'}(\mathbf{e}'_i) \equiv 0 \pmod{Q_1}$ for $i \leq n'$.

Recovering vectors from diagonal layer: The only task that remains is to find all the diagonal vectors. The attacker can apply Wolf et al.'s [50] trick to find all the diagonal vectors in the layer. Here observe that the computation of finding a vector in O_2 , dominates the computation of finding a vector in O_1 .

Attack Complexity. The complexity of the first step dominates the complexity of other steps involved in this algorithm.

- In the case of odd characteristic, its estimated cost is given by the complexity of solving a system of m random homogeneous quadratic equations in $n-m$ variables. If $n-m \leq m$, this computation can be performed via XL algorithm and it requires

$$3 \cdot q \binom{n-m-1+d}{d}^2 \binom{n-m+1}{2}$$

field operations, where d is the operating degree of the algorithm. It means, d -is the smallest positive integer so that the coefficient of t^d in the power series $(1-t^2)^m/(1-t)^{n-m}$ is non-positive.

- In the case of even characteristic, the complexity of the attack is slightly better. Its cost can be estimated by the complexity of solving a system of $m-1$ random homogeneous quadratic equations in $n-m-1$ variables. Assuming $n-m \leq m$, this can be solved with XL algorithm and requires:

$$3 \cdot q \binom{n-m-2+d}{d}^2 \binom{n-m}{2}$$

- In the case $n-m > m$ we estimate the complexity by the cost of solving an undetermined system of m quadratic equations in $n-m$ variables.

Example for $SL-1$ parameters. Apply Beullens' trick to guess a vector in O_2 , which happens with probability $1/q$. Finding one vector on O_2 asks to solve a quadratic system of 69 equations in 66 unknowns. This computation is the most costly in the entire algorithm. Solving this quadratic system needs 2^{157} field operations. The guessing needs $1/q$ search and cost of one $\mathbb{F}_{16}$ - multiplication needs 36 gates. Therefore, the overall complexity is about 2^{166} .

5.3 Rectangular Min-rank Attack on VDOO

Rectangular min-rank attack is proposed by Beullens [8]. We first describe the attack against VDOO and then compute the required attack complexity to perform this attack against VDOO. Attacker starts with $n \times m$ -rectangular matrices $M_1, M_2, \dots, M_n$ over $\mathbb{F}_q$ where each M_i is defined as

$$M_i = \begin{bmatrix} \mathcal{DP}(\mathbf{s}_1, \mathbf{s}_i) \\ \mathcal{DP}(\mathbf{s}_2, \mathbf{s}_i) \\ \vdots \\ \mathcal{DP}(\mathbf{s}_n, \mathbf{s}_i) \end{bmatrix}$$

where $(\mathbf{s}_i)_{i=1}^n$ is a basis of $\mathbb{F}_q^n$.

Let $\mathbf{o}_2 \in \mathbb{F}_q^n$. The bi-linearity of $\mathcal{DP}$ implies

$$M := \sum_{i=1}^n o_{2i} M_i := \begin{bmatrix} \mathcal{DP}(\mathbf{s}_1, \mathbf{o}_2) \\ \mathcal{DP}(\mathbf{s}_2, \mathbf{o}_2) \\ \vdots \\ \mathcal{DP}(\mathbf{s}_n, \mathbf{o}_2) \end{bmatrix}.$$

Hence, the maximum rank of M is o_2 , since $\dim(W) = o_2$ and $\mathcal{DP}(\mathbf{x}, O_2) \in W$ for every $\mathbf{x} \in \mathbb{F}_q^n$. This observation provides attacker a min-rank instance to find o_{2i} 's in $\mathbb{F}_q$.

To enhance the performance of the simple attack, Beullens combined the rectangular min-rank attack with the simple attack [9]. Like earlier, the attacker fixes $\mathbf{v}$ to get a linear map $\mathcal{DP}_{\mathbf{v}}$. This helps to find $\mathbf{o}_2 \in O_2$ using $\mathcal{DP}_{\mathbf{v}}(\mathbf{o}_2) = 0$.

This system of linear equations reduces the number of matrices by m in the rectangular min-rank instance. Thus, the basis of $\text{Ker}(\mathcal{DP}_{\mathbf{v}})$ is $\mathbf{b}_1, \dots, \mathbf{b}_{n-m}$. Hence, the new min-rank instance has $n-m$ matrices $\widetilde{M}_i$, where

$$\widetilde{M}_i := \sum_{j=1}^n b_{ij} M_j := \begin{bmatrix} \mathcal{DP}(\mathbf{s}_1, \mathbf{b}_i) \\ \mathcal{DP}(\mathbf{s}_2, \mathbf{b}_i) \\ \vdots \\ \mathcal{DP}(\mathbf{s}_n, \mathbf{b}_i) \end{bmatrix}, \quad \text{for } i = 1 \text{ to } n-m.$$

If $\mathbf{y}$ is a solution of the new min-rank problem having $n-m$ matrices then $\mathbf{o}_2 = \sum_{i=1}^{n-m} y_i \mathbf{b}_i$ is a solution of the old min-rank problem. Hence, the attack needs to be repeated approximately q times, until it finds $\mathbf{o}_2 \in \text{Ker}(\mathcal{DP}_{\mathbf{v}}) \cap O_2 \neq \{0\}$.

Attack Complexity. The cost of the attack can be estimated by the complexity of solving a min-rank instance with $n-m$ matrices of size $n-1 \times m$ and target rank o_2 . The number of field multiplications required to perform this attack is

$$3 \cdot q \cdot (n-m-1)(o_2+1) \binom{m}{o_2}^2 \cdot \binom{n-m+b-2}{b}^2$$

where b is the operating degree for the algorithm [4].

Example for SL-1 parameters. The attacker needs to guess a good $\mathcal{DP}_{\mathbf{x}}$. After then the attacker gets a min-rank instance of 98 matrices which has 137 rows and 60 columns and the span of these matrices has a matrix of rank 40. Bardet et al.'s [4] algorithm provides an efficient way to solve this min-rank instance. This computation needs 2^{162} -field operations.

5.4 Kipnis-Shamir Attack on VDOO

The attacker targeting VDOO can employ a technique similar to the one devised by Kipnis and Shamir [32] to retrieve the subspace O_2 . This approach effectively aids in the separation of oil and vinegar variables, ultimately leading to the recovery of the private key. The complexity of this attack can be roughly estimated as $O(o_2^4 \cdot q^{n-o_2-1})$ field multiplications. To expedite this assault, the attacker leverages Grover's algorithm, which serves to reduce the complexity to $O(o_2^4 \cdot q^{(n-o_2-1)/2})$.

Example for SL-1 parameters. Attacker needs to perform approximately 2^{250} -field operations in classical settings and 2^{138} -field operations in quantum computer.

5.5 Intersection Attack on VDOO

Beullens introduced the intersection attack [8], which effectively reduced the claimed security level of the Rainbow signature scheme by approximately 20 bits compared to the original design. In this attack, Beullens improved upon the Rainbow band separation attack [16] using the analysis proposed by Perlner [41]. The intersection attack helps to identify k -vectors simultaneously within the oil-space O_2 by solving a system of quadratic equations for a vector within the intersection $\cap_{i=1}^k L_i O_2$, where L_i 's are invertible matrices. This attack performs well when the intersection is non-empty, which occurs when $n < \frac{2k-1}{k-1} o_2$. An upper bound of the computational cost of this attack is given by

$$3\overline{\mathcal{M}}(A,B)^2 \cdot \binom{n_x+2}{2},$$

where $n_x = \min(nk - (2k-1)o_2, n-1)$ and $\overline{\mathcal{M}}(A,B)$ is defined in [8]. However, in the case of VDOO where $n \geq 3o_2$, there is no guarantee that the subspace (for more details, see [8]) namely $L_i O_2 \cap L_j O_2$ will exist. Consequently, the attack becomes probabilistic for VDOO and will succeed with a probability of $\frac{1}{q^{(n-3o_2+1)}}$.

Example for SL-1 parameters. The complexity to break SL-1 parameters, attacker needs 2^{227} -field multiplications.

5.6 Quantum Attacks

The attacker can accelerate certain aspects of the classical attacks using a quantum computer. For MQ- or OV-based schemes the only quantum algorithm that can help in cryptanalysis is Grover's search [28]. This algorithm reduces the search space, thereby reducing the number of field multiplications by a factor $q^{k/2}$. Grover's can be combined with a hybrid attack, leading to a slight speed-up than simple Grover [5, 21], but this specifically does not threaten the post-quantum security of our scheme [12].

Recently, [36] proposed a quantum algorithm to solve underdetermined systems of homogeneous quadratic equations. It enhances the algorithm [12] by combining the exhaustive search from [25] and an improvement of the linearisation technique from [29]. This algorithm transforms in polynomial time the undetermined system of m equations in n variables into a new system of equations which has:

- p equations in one variable that can be solved sequentially.
- $m-k-p$ linearised equations.
- k remaining equations.

To be able to make this transformations, we need:

$$n-1 \geq (m-k-p-1)(m-k+2) \quad \text{and} \quad n-m \geq \frac{p(2m-2k-p-1)}{2}. \quad (7)$$

The authors of this algorithm remarks that it is specially well-suited for moderately underdetermined systems with $\frac{2n}{m^2+m}$ large because of the flexibility in choosing the parameters p and k .

The runtime of the algorithm is dominated by the runtime of *Full-Just-Guess-Solver* step. Given k, p satisfying (7), a lower bound of the runtime needed for solving this step is given by:

$$\begin{aligned} & p \cdot q^k \text{ classically, and} \\ & p \cdot q^{(k/2)} \text{ quantumly.} \end{aligned}$$

By [36, Remark 4] this complexity is minimized for the smallest k that satisfies (7).

5.7 Provable security: EUF-CMA Security

Our VDOO scheme, similar to UOV, Rainbow, and other UOV-based signature schemes, offers universal unforgeability [15]. Like these other schemes, we incorporate a salt in the signature generation process to demonstrate the EUF-CMA security of our scheme. We have followed the established methodology for this purpose, as seen in prior work such as [44, 10]. Here, we have only provided an outline of the proof. The full proof can be done using similar strategies as Mayo [10], QR-UOV [23], PROV [20], etc. Our security proof relies on the well-understood hardness of the UOV problem. We begin by defining the UOV problem and then introduce the VDOO problem.

For security reasons, we recommend that each salt value should be used for no more than one signature. Consequently, we fix the salt length at 16 bytes, assuming up to 2^{64} signature generations within the system [12].

UOV Problem. Suppose $\text{UOV}_{(n,v,o,q)}$ denotes a family of UOV public polynomial maps where n is the number of variables, $v+o$ is the number of equations, and q is the size of the finite field, and $\text{MQ}_{(q,n,m)}$ denotes a family of random quadratic systems with n unknowns and m constraints over $\mathbb{F}_q$. The UOV problem asks to distinguish $\mathcal{P}$ from $\text{UOV}_{(q,n,v,o)}$ and $\text{MQ}_{(q,n,m)}$. Suppose $\mathcal{A}_{\text{UOV}}$ is an adversary that solves the distinguishing problem and has distinguishing advantage

$$\text{Adv}_{\text{UOV}}(\mathcal{A}_{\text{UOV}}) = |\Pr[\mathcal{A}_{\text{UOV}}(\mathcal{P}) = 1 \mid \mathcal{P} \in \text{MQ}] - \Pr[\mathcal{A}_{\text{UOV}}(\mathcal{P}) = 1 \mid \mathcal{P} \in \text{UOV}]|.$$

It is widely believed that there is no probabilistic polynomial-time adversary, including quantum adversaries, denoted as $\mathcal{A}$, that can efficiently solve the UOV problem.

VDOO Problem. Suppose VDOO is a family of VDOO public polynomial maps. Given a random $\mathcal{P} \in \text{VDOO}$ and $\mathbf{t} \in \mathbb{F}_q^m$, the VDOO problem asks to find $\mathbf{s}$ such that $\mathcal{P}(\mathbf{s}) = \mathbf{t}$. If $\mathcal{A}$ is such an adversary that computes the inverse of the VDOO public map, then the advantage of this computation is

$$\text{Adv}_{\text{VDOO}}(\mathcal{A}_{\text{VDOO}}) = \Pr[\mathcal{P}(\mathbf{s}) = \mathbf{t} \mid \mathcal{P} \in \text{VDOO}, \mathcal{A}_{\text{VDOO}}(\mathcal{P}, \mathbf{t}) = \mathbf{s}].$$

Now we are going to state our main security claim, which establishes the EUF-CMA security of VDOO. To understand the security notion, we refer to [10, 44, 33].

Main security claim. Suppose the adversary $\mathcal{A}$ runs in time T to solve the EUF-CMA game of VDOO in the random oracle model. This adversary makes q_s signing queries and q_h random oracle queries. Then there exist $\mathcal{A}_{\text{UOV}}$ and $\mathcal{A}_{\text{VDOO}}$ running in time $T + O((q_s + q_t) \cdot \text{poly}(q, v, d, o_1, o_2))$ with

$$\text{Adv}_{\text{VDOO}}^{\text{EUF-CMA}}(\mathcal{A}) \leq \text{Adv}_{\text{UOV}_{(q,v',o')}}(\mathcal{A}_{\text{UOV}}) + q_h \cdot \text{Adv}_{\text{VDOO}_{(q,v,d,o_1,o_2)}}(\mathcal{A}_{\text{VDOO}}) + (q_s + q_h)q_s \cdot 2^{-|\text{salt}|} + q^{-m}.$$

Proof idea. Here, we informally sketch the proof. We can adopt the proof methodology used in Mayo (see theorem 6 from [10]). In the first step, we can establish a reduction from the EUF-CMA security of the VDOO signature scheme to EUF-KOA (Existential unforgeability against key-only attack) security by simulating the signing oracle. Note that, the adversary does not have access to the signing oracle in the EUF-KOA game. Once this reduction is established, we can easily show a reduction from the

UOV problem and VDOO problem to the EUF-KOA security game in the second step. Like the security proof of Mayo [10], we can use the hybrid proof system to establish both reductions. This proof style has also been adopted by many state-of-the-art OV-based constructions [23, 24, 20, 14]. Finally, we can combine both of these two steps to establish the above theorem.

5.8 Other Security Properties

Side-channel Security. The VDOO signature scheme preserves the computational structure of UOV during the signing procedure, differing only in the design of the central map. Consequently, VDOO inherits the mathematical structure of UOV-type multivariate signature schemes. A comprehensive systematization of implementation-level attacks against multivariate signatures, including timing, power, electromagnetic, and fault attacks, together with their corresponding countermeasures, has recently been presented by Aulbach *et al.* [1, 34]. Since the signing algorithm of VDOO follows the same computational paradigm as UOV, the observations and attack models discussed in their work are directly relevant to the implementation of VDOO.

Likewise, implementation-level countermeasures developed for UOV-type signature schemes can be readily adopted for VDOO [34]. In particular, masking techniques for Gaussian elimination [38] and existing masking schemes for UOV-type signature generation can also be easily adopted for VDOO.

6 Parameters Selection

In this section, we present our chosen parameters based on the algebraic security analysis. We assess the *practicality* of the VDOO signature scheme, which entails a fine-tuned trade-off among computation time, security, and communication costs. Table 1 explicitly presents the signature, private key, and public key sizes for the VDOO scheme at the 128-bit, 256-bit, and 512-bit classical security levels, with parameters selected according to the NGCC recommendations and defined by the tuple (q, v, d, o_1, o_2) . We consider the complexity of two primary attacks: the simple attack [9] (SA) and the rectangular min-rank attack [8] (RA). From the attacker’s point of view, these two attacks exhibit the most optimistic complexity among all other known attacks. Here, the complexity represents the number of field multiplications required for their execution.

Table 1: VDOO parameter sets, key sizes, signature sizes, and estimated security against signature attacks (SA) and rank attacks (RA).

Security level	params (q, v, d, o_1, o_2)	Signature size (B)	Private key size (KB)	Public key size (KB)	Attacks (SA, RA)
128-bit	(16, 67, 16, 14, 40)	85	226.7	323.1	(166, 162)
256-bit	(256, 205, 10, 17, 68)	316	3529.0	4188.7	(278, 281)
512-bit	(256, 260, 10, 10, 175)	471	15882.9	19755.2	(526, 642)

7 Implementation Details

7.1 Finite Field Representation

Representation of $\mathbb{F}_{16}$. Our implementation adopts a tower-field representation in which $\mathbb{F}_4 = \mathbb{F}_2[\alpha]/(\alpha^2 + \alpha + 1)$, and $\mathbb{F}_{16} = \mathbb{F}_4[\beta]/(\beta^2 + \beta + \alpha)$. Elements of $\mathbb{F}_2$ are represented by a single bit, whereas elements of $\mathbb{F}_4$ are stored as degree-one polynomials over $\mathbb{F}_2$ using two bits, with the constant coefficient occupying the least significant bit. An element of $\mathbb{F}_{16}$ is represented as a degree-one polynomial over $\mathbb{F}_4$ and stored in four bits, where the constant $\mathbb{F}_4$ coefficient occupies the least significant two bits. Two adjacent $\mathbb{F}_{16}$ elements are packed into a single byte, with the lower nibble preceding the upper nibble. Field addition is performed by bitwise XOR, while multiplication, squaring, and inversion are implemented recursively using the tower-field construction. This representation naturally supports efficient constant-time arithmetic and SIMD-friendly vectorized implementations without requiring large lookup tables.

Representation of $\mathbb{F}_{256}$. The field $\mathbb{F}_{256}$ is constructed as the quadratic extension $\mathbb{F}_{256} = \mathbb{F}_{16}[\gamma]/(\gamma^2 + \gamma + \alpha\beta)$, where $\alpha\beta$ denotes the multiplication of the generators of $\mathbb{F}_4$ and $\mathbb{F}_{16}$. Each field element is represented as a degree-one polynomial over $\mathbb{F}_{16}$ and stored in a single byte, with the constant $\mathbb{F}_{16}$ coefficient occupying the lower four bits and the linear coefficient occupying the upper four bits. Arithmetic over $\mathbb{F}_{256}$ is realized recursively using the underlying $\mathbb{F}_{16}$ operations together with Karatsuba multiplication, requiring only three $\mathbb{F}_{16}$ multiplications followed by reduction modulo the defining polynomial. This recursive tower-field representation provides compact storage, constant-time arithmetic, and efficient vectorized implementations for the key generation, signing, and verification algorithms.

7.2 Symmetric Primitives

The implementation employs the reference software provided by the Institute of Commercial Cryptography Standards (ICCS) as part of the *Next-generation Commercial Cryptographic Algorithms Program* (NGCC) for all symmetric cryptographic primitives. In particular, the implementation uses the SM3 hash function together with the accompanying HMAC, pseudohash, and pseudo-XOF subroutines supplied by the ICCS reference library. In addition, all deterministic pseudorandom byte generation is performed using the ICCS deterministic random number generator (DRNG), which is initialized from a seed and subsequently used to generate the pseudorandom values required during key generation and other randomized procedures. Since the focus of this work is the efficient implementation of the VDOO signature scheme, these cryptographic components are treated as black-box primitives, and the corresponding ICCS reference implementations are used without modification.

7.3 Key Generation

The key generation procedure implements Algorithm 3 from Section 4. We describe here the principal implementation choices made in the reference code.

Sampling invertible affine maps. The affine maps $\mathcal{S}: \mathbb{F}_q^m \rightarrow \mathbb{F}_q^m$ and $\mathcal{T}: \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$ are each required to be invertible. The reference implementation generates each map via an LU decomposition through `generate_M_invM`: a lower triangular matrix L and an upper triangular matrix U are sampled independently over $\mathbb{F}_q$, with nonzero diagonal entries enforced by construction. The product $M = LU$ is computed by `multiply_L_U`, and its inverse $M^{-1} = U^{-1}L^{-1}$ is obtained by composing the outputs of

`invert_upper_triangular` and `invert_lower_triangular` via `multiply_invU_invL`, without any matrix inversion of a general dense matrix. Once the public key is assembled, the transient factors S and T are zeroized; only S^{-1} and T^{-1} are retained in the secret key via `generate_private_key`, as these are the quantities required during signing by Algorithm 4.

Generating the central map. The central map $\mathcal{F}=(f_1,\dots,f_m)$ is initialized by `generate_central_map`, which fills all coefficient arrays with uniform random field elements via `get_randombytes`. A subsequent pass through `set_zero_forbidden_terms` enforces the VDOO layer structure defined in Section 4 by zeroing all forbidden monomials: the predicate `is_term_allowed` dispatches to `is_allowed_diag`, `is_allowed_oil1`, or `is_allowed_oil2` depending on the layer, zeroing any monomial $x_i x_j$ not of the form `vinegar` $\times$ `vinegar` or `vinegar` $\times$ x_k in the diagonal layer, and zeroing all `oil` $\times$ `oil` cross terms in the two oil layers following the standard UOV constraint.

Assembling the public key. The public key $\mathcal{P}=\mathcal{S}\circ\mathcal{F}\circ\mathcal{T}$ is computed in two stages by `generate_public_key`. First, `apply_T_to_F` evaluates the composition $\mathcal{F}\circ\mathcal{T}$ by applying $\mathcal{T}$ to each polynomial in $\mathcal{F}$ via the identity $f_k(\mathcal{T}\mathbf{w})=\mathbf{w}^\top(T^\top A_k T)\mathbf{w}$, where A_k is the symmetric matrix representing the homogeneous quadratic part of f_k ; the resulting coefficient matrices are triangularized and stored. Second, `apply_S_to_FT` applies $\mathcal{S}$ by left-multiplying the coefficient blocks of $\mathcal{F}\circ\mathcal{T}$, yielding the final public key coefficients. Both stages exploit batched matrix operations over $\mathbb{F}_q$ to process all m polynomials simultaneously, before the intermediate buffer is zeroized by `generate_keypair`.

7.4 Signature Generation

The signature generation procedure implements Algorithm 4 from Section 4. We describe here the principal implementation choices made in the reference code.

Hash and salt. The top-level routine `vdoo_sign` samples a uniform random salt of `SALT_BYTES` bytes via `get_randombytes` and computes the target vector $\mathbf{d}=\mathcal{H}(\mathcal{H}(\text{msg})\|\text{salt})\in\mathbb{F}_q^m$ via `hash_msg`. The double hashing avoids reprocessing a long message on each retry attempt. A fresh salt is drawn on each failed inversion attempt; at most 100 attempts are made before returning a failure code.

Applying $\mathcal{S}^{-1}$. The target vector $\mathbf{d}\in\mathbb{F}_q^m$ is mapped into the domain of the central map by `apply_invS`, which takes S^{-1} and $\mathbf{d}$ as input and returns $\mathbf{t}=\mathcal{S}^{-1}(\mathbf{d})\in\mathbb{F}_q^m$ as a dense matrix-vector product over $\mathbb{F}_q$.

Inverting the central map. The inversion of $\mathcal{F}$ is delegated to `vdoo_inverse_F`, which takes the central map $\mathcal{F}$, the secret key `sk`, and the target vector $\mathbf{t}\in\mathbb{F}_q^m$ as input and returns a preimage $\mathbf{y}\in\mathbb{F}_q^n$ satisfying $\mathcal{F}(\mathbf{y})=\mathbf{t}$, realizing Algorithm 2. Internally, the diagonal equations are resolved by `solve_diagonal_eq`, and the two oil layers are handled by `substitute_fixed_vars` followed by `gauss_elimination`, each operating on a square linear system of size o_1 and o_2 respectively. If any layer yields a failure, `vdoo_inverse_F` restarts with freshly sampled vinegar variables; at most 100 attempts are made.

Applying $\mathcal{T}^{-1}$. Once the preimage $\mathbf{y}\in\mathbb{F}_q^n$ is found, `apply_invT` takes T^{-1} and $\mathbf{y}$ as input and returns $\mathbf{s}=\mathcal{T}^{-1}(\mathbf{y})\in\mathbb{F}_q^n$ as a dense matrix-vector product over $\mathbb{F}_q$, analogous to `apply_invS`. The final signature returned by `vdoo_sign` is the concatenation $\sigma=(\mathbf{s},\text{salt})$.

7.5 Verification

The verification procedure implements Algorithm 5 from Section 4, following the same hash recomputation strategy as `vdoo_sign`. The evaluation of the public map $\mathcal{P}(\mathbf{s})$ is performed by `vdoo_optimized_evaluation`, which takes the public key $\mathcal{P}$ and the signature vector $\mathbf{s} \in \mathbb{F}_q^n$ as input and returns $\mathbf{d}' = \mathcal{P}(\mathbf{s}) \in \mathbb{F}_q^m$ using an optimized monomial batching strategy that reduces the number of field multiplications compared to a straightforward evaluation of the public map.

7.6 NGCC API Functions

The reference implementation conforms to the standard NGCC signature API, which exposes three interfaces for key generation, signature generation, and signature verification. These APIs provide a uniform abstraction for invoking the VDOO signature scheme while hiding the internal implementation details.

VDOO Key Generation API: `crypto_sign_keypair`. This function generates a public/secret key pair. The output buffer `pk` stores the serialized public key, while `sk` stores the serialized secret key. Internally, the routine samples a uniformly random secret seed using the NGCC deterministic random number generator, invokes `generate_keypair()` to construct the affine transformations and the structured central map, and derives the corresponding public key. The temporary seed is securely erased before returning.

VDOO Signature Generation API: `crypto_sign`. This function produces a digital signature. The inputs are the message `m` of length `milen` bytes and the secret key `sk`. The outputs are the signed message `sm`, consisting of the original message concatenated with the signature, and its total length `smilen`. The implementation first hashes the input message and then invokes the internal `vdoo_sign()` routine to generate the signature.

VDOO Verification API: `crypto_sign_open`. The verification routine takes as input a signed message `sm` of length `smilen` and the public key `pk`. It separates the message from the appended signature, recomputes the message digest, and invokes the internal `vdoo_verify()` routine. If verification succeeds, the recovered message is written to `m`, its length is returned in `milen`, and the function returns success; otherwise, an error code indicating verification failure is returned.

7.7 Known Answer Test Values

The submission includes Known Answer Test (KAT) files containing tuples of secret keys (`sk`), public keys, signatures, messages, and seedS for each implementation tier: VDOO-128, VDOO-256, and VDOO-512. These validation files are located within the `Test_Vector` folders of the submission package.

8 Performance of VDOO

The performance evaluation of the VDOO signature scheme was conducted on a mobile workstation equipped with an AMD Ryzen 7 PRO 8840HS processor featuring 16 logical cores and a base clock frequency of 3.3 GHz (with a typical operating frequency of 2.6 GHz during the benchmark tests). The system is supported by 16.0 GiB of RAM and runs on Ubuntu 22.04.5 LTS (64-bit) using the Wayland windowing system. All implementations were written in C and compiled using the gcc 11.4.0 compiler infrastructure with standard optimization flags (`-O2` for the reference implementation and `-O3` for the optimized

Table 2: Performance comparison of the reference and optimized implementations of VDOO. We report the median (Med) and average (Avg) CPU cycle counts for key generation, signing, and verification, measured across 100 independent executions (except for the VDOO-256 and VDOO-512 reference implementations, which were executed 10 and 1 times, respectively).

Parameter	Impl.	KeyGen		Sign		Verify	
		Med	Avg	Med	Avg	Med	Avg
VDOO-128	Ref	991,315,660	994,185,163	122,165,736	200,462,507	1,724,002	1,790,838
	Opt	194,375,620	195,221,084	120,984,230	198,504,614	1,702,115	1,765,903
VDOO-256	Ref	32,803,517,208	32,726,290,997	325,629,447	406,475,704	7,464,039	7,448,713
	Opt	2,928,885,465	2,931,402,808	320,911,684	401,200,352	7,291,402	7,315,884
VDOO-512	Ref	262,545,499,722	262,545,499,722	5,583,275,544	5,583,275,544	20,681,991	20,681,991
	Opt	14,505,276,228	14,510,992,375	5,520,414,906	5,560,998,204	20,324,771	20,401,338

implementation as required by the benchmark guidelines) to leverage modern instruction sets. Performance metrics, specifically clock cycles (ticks), were captured using the processor’s hardware performance counters to ensure high-precision measurement across the key generation, signing, and verification phases. Each operation was executed 100 times, and the median and average execution times are reported in Table 2.

8.1 Results

Table 2 summarizes the performance of both the reference and optimized implementations of VDOO for the three parameter sets VDOO-128, VDOO-256, and VDOO-512. For each implementation, we report the median and average CPU cycle counts for key generation, signing, and verification, measured over 100 independent executions (except for the VDOO-256 and VDOO-512 reference implementations, which were executed 10 and 1 times, respectively). Since the proposed optimization only modifies the key-generation procedure, the signing and verification algorithms remain unchanged, allowing a direct comparison of the computational benefits achieved by the optimized implementation.

8.2 Discussion

The experimental results demonstrate that the proposed optimization substantially accelerates key generation while leaving the signing and verification procedures essentially unchanged. For VDOO-128, the optimized implementation reduces the median key-generation cost from 991,315,660 to 194,375,620 clock cycles, corresponding to an 80.4% reduction in execution time, or equivalently a $5.10\times$ speedup. At the VDOO-256 security level, the optimization lowers the median cost from 32.80×10^9 to 2.93×10^9 clock cycles, achieving a 91.1% reduction ($11.2\times$ speedup). The improvement is even more pronounced for VDOO-512, where the median key-generation cost decreases from 262.55×10^9 to 14.51×10^9 clock cycles, corresponding to a 94.5% reduction ($18.1\times$ speedup).

In contrast, the median signing time improves by only 0.97%, 1.45%, and 1.13% for VDOO-128, VDOO-256, and VDOO-512, respectively, while the median verification time improves by 1.27%, 2.31%, and 1.73%. These modest differences are expected because the optimization exclusively targets the efficient construction of the public key by exploiting the block structure of the central map and the structured linear transforma-

tions. The signing algorithm, which is dominated by the inversion of the central map, and the verification algorithm, which evaluates the public map, remain algorithmically identical in both implementations.

An important observation is that the benefit of the optimized key-generation algorithm increases with the security level. As the dimensions of the central map grow, the computational overhead of the straightforward matrix-based construction increases rapidly, whereas the proposed block-wise algorithm avoids a significant fraction of redundant matrix operations. Consequently, the relative improvement rises from 80.4% at the 128-bit security level to 94.5% at the 512-bit security level, demonstrating the excellent scalability of the proposed optimization for high-security parameter sets.

8.3 Limitations

- *Large Public Key Sizes:* Multivariate schemes need very large public keys (often over 100 KB), which makes them much harder to store and send compared to other post-quantum methods like Dilithium.
- *High Computational Cost:* Although signing and verification avoid expensive number-theoretic operations, they still require evaluating and solving large systems of multivariate quadratic equations over finite fields. Consequently, signature generation and verification consume a significant number of CPU cycles, particularly for higher security parameter sets.
- *VDOO Problem is less studied:* While some methods have been around for a long time, newer designs like VDOO haven't been studied as much by the global research community, meaning we aren't as sure about their long-term security.

References

- [1] Thomas Aulbach, Fabio Campos, and Juliane Krämer. Sok: On the physical security of uov-based signature schemes. In Ruben Niederhagen and Markku-Juhani O. Saarinen, editors, *Post-Quantum Cryptography*, pages 199–231, Cham, 2025. Springer Nature Switzerland.
- [2] Jean-Philippe Aumasson, Daniel J. Bernstein, Ward Beullens, Christoph Dobraunig, Maria Eichlseder, Scott Fluhrer, Stefan-Lukas Gazdag, Andreas Hülsing, Panos Kampanakis, Stefan Kölbl, Tanja Lange, Florian Mendel, Martin M. Lauridsen, Ruben Niederhagen, Christian Rechberger, Joost Rijneveld, Peter Schwabe, and Bas Westerbaan. Sphincs+ submission to the nist post-quantum project, v.3.1, 2018. [Online; accessed 10-June-2023].
- [3] Magali Bardet, Maxime Bros, Daniel Cabarcas, Philippe Gaborit, Ray Perlner, Daniel Smith-Tone, Jean-Pierre Tillich, and Javier Verbel. Algebraic attacks for solving the rank decoding and min-rank problems without Gröbner basis (2020). *Preprint available on <https://arxiv.org/pdf/2002.08322.pdf>*, 3:22–30, 2002.
- [4] Magali Bardet, Maxime Bros, Daniel Cabarcas, Philippe Gaborit, Ray Perlner, Daniel Smith-Tone, Jean-Pierre Tillich, and Javier Verbel. Improvements of algebraic attacks for solving the rank decoding and MinRank problems. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 507–536. Springer, 2020.
- [5] Daniel J. Bernstein and Bo-Yin Yang. Asymptotically faster quantum algorithms to solve multivariate quadratic equations. *IACR Cryptol. ePrint Arch.*, 2017:1206, 2017.
- [6] Luk Bettale, Jean-Charles Faugère, and Ludovic Perret. Hybrid approach for solving multivariate systems over finite fields. *Journal of Mathematical Cryptology*, 3(3):177–197, 2009.
- [7] Luk Bettale, Jean-Charles Faugère, and Ludovic Perret. Solving polynomial systems over finite fields: improved analysis of the hybrid approach. In Joris van der Hoeven and Mark van Hoeij, editors, *International Symposium on Symbolic and Algebraic Computation, ISSAC’12, Grenoble, France - July 22 - 25, 2012*, pages 67–74. ACM, 2012.
- [8] Ward Beullens. Improved cryptanalysis of UOV and Rainbow. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 348–373. Springer, 2021.
- [9] Ward Beullens. Breaking Rainbow takes a weekend on a laptop. *Cryptology ePrint Archive*, 2022.
- [10] Ward Beullens. Mayo: practical post-quantum signatures from oil-and-vinegar maps. In *Selected Areas in Cryptography: 28th International Conference, Virtual Event, September 29–October 1, 2021, Revised Selected Papers*, pages 355–376. Springer, 2022.
- [11] Antoine Casanova, Jean-Charles Faugère, Gilles Macario-Rat, Jacques Patarin, Ludovic Perret, and Jocelyn Ryckeghem. *GeMSS: A Great Multivariate Short Signature*. PhD thesis, UPMC-Paris 6 Sorbonne Universités; INRIA Paris Research Centre, MAMBA Team . . . , 2017.
- [12] Lily Chen, D Moody, and YK Liu. NIST post-quantum cryptography standardization. *Transition*, 800:131A, 2017.
- [13] Nicolas Courtois, Alexander Klimov, Jacques Patarin, and Adi Shamir. Efficient algorithms for solving overdefined systems of multivariate polynomial equations. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 392–407. Springer, 2000.

- [14] Jintai Ding. Tuov: Triangular unbalanced oil and vinegar. ., 2023.
- [15] Jintai Ding and Dieter Schmidt. Rainbow, a new multivariable polynomial signature scheme. In *International conference on applied cryptography and network security*, pages 164–175. Springer, 2005.
- [16] Jintai Ding, Bo-Yin Yang, Chia-Hsin Owen Chen, Ming-Shing Chen, and Chen-Mou Cheng. New differential-algebraic attacks and reparametrization of Rainbow. In *International Conference on Applied Cryptography and Network Security*, pages 242–257. Springer, 2008.
- [17] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1):238–268, Feb. 2018.
- [18] Jean-Charles Faugere. A new efficient algorithm for computing Gröbner bases (F4). *Journal of pure and applied algebra*, 139(1-3):61–88, 1999.
- [19] Jean Charles Faugere. A new efficient algorithm for computing Gröbner bases without reduction to zero (F5). In *Proceedings of the 2002 International Symposium on Symbolic and Algebraic Computation*, pages 75–83, 2002.
- [20] Jean-Charles Faugere, Pierre-Alain Fouque, Gilles Macario-Rat, Brice Minaud, and Jacques Patarin. PROV: PProvable unbalanced Oil and Vinegar specification v1. 0–06/01/2023. ., 2023.
- [21] Jean-Charles Faugère, Kelsey Horan, Delaram Kahrobaei, Marc Kaplan, Elham Kashefi, and Ludovic Perret. Fast quantum algorithm for solving multivariate quadratic equations. *CoRR*, abs/1712.07211, 2017.
- [22] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. Falcon: Fast-fourier lattice-based compact signatures over ntru, 2018. [Online; accessed 10-June-2023].
- [23] Hiroki Furue, Yasuhiko Ikematsu, Fumitaka Hoshino, Yutaro Kiyomura, Tsunekazu Saito, and Tsuyoshi Takagi. Qr-uov. ., 2023.
- [24] Hiroki Furue, Yasuhiko Ikematsu, Yutaro Kiyomura, and Tsuyoshi Takagi. A new variant of unbalanced oil and vinegar using quotient ring: QR-UOV. In *Advances in Cryptology–ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part IV 27*, pages 187–217. Springer, 2021.
- [25] Hiroki Furue, Shuhei Nakamura, and Tsuyoshi Takagi. Improving thomae-wolf algorithm for solving underdetermined multivariate quadratic polynomial problem. In *International Conference on Post-Quantum Cryptography*, pages 65–78. Springer, 2021.
- [26] Anindya Ganguly, Angshuman Karmakar, and Nitin Saxena. Vdoo: a short, fast, post-quantum multivariate digital signature scheme. In *International Conference on Cryptology in India*, pages 197–222. Springer, 2023.
- [27] Google Groups. Rainbow round3 official comment, 2022.
- [28] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM Symposium on Theory of Computing*, pages 212–219, 1996.
- [29] Yasufumi Hashimoto. An improvement of algorithms to solve under-defined systems of multivariate quadratic equations. *Cryptology ePrint Archive*, 2021.

- [30] David S Johnson and Michael R Garey. *Computers and Intractability: A Guide to the Theory of NP-completeness*. WH Freeman, 1979.
- [31] Aviad Kipnis, Jacques Patarin, and Louis Goubin. Unbalanced oil and vinegar signature schemes. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 206–222. Springer, 1999.
- [32] Aviad Kipnis and Adi Shamir. Cryptanalysis of the Oil and Vinegar signature scheme. In *Annual international cryptology conference*, pages 257–266. Springer, 1998.
- [33] Haruhisa Kosuge and Keita Xagawa. Probabilistic hash-and-sign with retry in the quantum random oracle model. *Cryptology ePrint Archive*, 2022.
- [34] Suparna Kundu, Quinten Norga, Angshuman Karmakar, Uttam Kumar Ojha, Anindya Ganguly, and Ingrid Verbauwhede. mUOV: Masking the unbalanced oil and vinegar digital signature scheme at first- and higher-order. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS ’25*, page 1994–2008, New York, NY, USA, 2025. Association for Computing Machinery.
- [35] Tsutomu Matsumoto and Hideki Imai. Public quadratic polynomial-tuples for efficient signature-verification and message-encryption. In *Workshop on the Theory and Application of Cryptographic Techniques*, pages 419–453. Springer, 1988.
- [36] Alexander May, Massimo Ostuzzi, and Henrik Ressler. Just guess: Improved (quantum) algorithm for the underdetermined mq problem. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 334–362. Springer, 2026.
- [37] T Moh. A public key system with signature and master key functions. *Communications in Algebra*, 27(5):2207–2222, 1999.
- [38] Quinten Norga, Suparna Kundu, Uttam Kumar Ojha, Anindya Ganguly, Angshuman Karmakar, and Ingrid Verbauwhede. Masking Gaussian Elimination at Arbitrary Order with Application to Multivariate-and Code-Based PQC. In Arpita Patra, editor, *Topics in Cryptology – CT-RSA 2025*, pages 249–272, Cham, 2025. Springer Nature Switzerland.
- [39] Jacques Patarin. Hidden fields equations (HFE) and isomorphisms of polynomials (IP): Two new families of asymmetric algorithms. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 33–48. Springer, 1996.
- [40] Jacques Patarin. The Oil and Vinegar signature scheme. In *Dagstuhl Workshop on Cryptography September 1997*, 1997.
- [41] Ray Perlner and Daniel Smith-Tone. Rainbow band separation is better than we thought. *Cryptology ePrint Archive*, 2020.
- [42] Albrecht Petzoldt, Stanislav Bulygin, and Johannes Buchmann. CyclicRainbow—a multivariate signature scheme with a partially cyclic public key. In *International Conference on Cryptology in India*, pages 33–48. Springer, 2010.
- [43] Albrecht Petzoldt, Stanislav Bulygin, and Johannes Buchmann. Selecting parameters for the Rainbow signature scheme. In *International Workshop on Post-Quantum Cryptography*, pages 218–240. Springer, 2010.

- [44] Koichi Sakumoto, Taizo Shirai, and Harunaga Hiwatari. On provable security of UOV and HFE signature schemes against chosen-message attack. In *International Workshop on Post-Quantum Cryptography*, pages 68–82. Springer, 2011.
- [45] Adi Shamir. Efficient signature schemes based on birational permutations. In *Annual International Cryptology Conference*, pages 1–12. Springer, 1994.
- [46] Daniel Smith-Tone, Ray Perlner, et al. Rainbow band separation is better than we thought. , 2020.
- [47] Chengdong Tao, Adama Diene, Shaohua Tang, and Jintai Ding. Simple matrix scheme for encryption. In *International Workshop on Post-Quantum Cryptography*, pages 231–242. Springer, 2013.
- [48] Enrico Thomae. A generalization of the rainbow band separation attack and its applications to multivariate schemes. *Cryptology ePrint Archive*, 2012.
- [49] Douglas Wiedemann. Solving sparse linear equations over finite fields. *IEEE transactions on information theory*, 32(1):54–62, 1986.
- [50] Christopher Wolf, An Braeken, and Bart Preneel. On the security of stepwise triangular systems. *Designs, Codes and Cryptography*, 40(3):285–302, 2006.
- [51] Bo-Yin Yang and Jiun-Ming Chen. Building secure tame-like multivariate public-key cryptosystems: The new TTS. In *Australasian Conference on Information Security and Privacy*, pages 518–531. Springer, 2005.